Predicción de fraude en transacciones

Comprensión del negocio, preparación de datos, comparación de algoritmos, evaluación temporal y aplicación web para consumir el modelo.

TRANSACCIONES	FRAUDES DETECTADOS (PRUEBA)	FALSOS POSITIVOS
1.362.621	**2.165 / 2.168**	**0**

Autor	Ezequiel Diaz Fernandez
Modelo final	Gradient Boosting (HistGB)
Aplicación	https://caece-aprendizaje-artificial-ezequiel.onrender.com
Repositorio	https://github.com/ezediazf/tpfinal
Fecha	23/10/2026

Resumen ejecutivo

El objetivo del trabajo es construir un modelo que estime si una transacción es fraudulenta (*isFraud*) a partir del dataset *Fraud.csv* (1.362.621 transacciones simuladas en 743 horas, 8.213 fraudes). Se siguió la metodología CRISP-DM: comprensión del negocio, auditoría y análisis exploratorio de los datos, ingeniería de variables, validación temporal, comparación de cuatro algoritmos contra una regla base, análisis crítico de los resultados y despliegue en una aplicación web.

> **Resultado.** El modelo elegido, **Gradient Boosting (HistGB)**, detecta en el período de prueba (últimos 8 días, nunca vistos durante el entrenamiento) **2.165 de 2.168 fraudes** (recall 99,86 %) con **0 falsos positivos** sobre 199.879 operaciones legítimas, y evita el 99,77 % del monto defraudado. Sólo usa información disponible *antes* de autorizar la operación.

Hallazgos principales:

- El fraude ocurre únicamente en **TRANSFER** y **CASH_OUT**, y en el 97,6 % de los casos **vacía exactamente la cuenta de origen**. Una regla basada sólo en ese patrón ya alcanza un recall de 97,0 % con precisión 100,0 %; se usó como línea base que todo modelo debía superar.
- El archivo provisto **no es una muestra uniforme**: contiene el tramo final del simulador más todos los fraudes anteriores. Los steps 1–352 sólo tienen fraudes. La tasa real en la ventana con ambas clases es 0,29 % (no 0,60 %). Se controló que el modelo no aproveche este artefacto.
- Random Forest y Gradient Boosting (HistGB) empataron en PR-AUC; se eligió Gradient Boosting (HistGB) porque es el más **robusto** cuando se le quitan las variables de vaciado (PR-AUC 0,9993 contra 0,9972), además de entrenar en 2,5 s.
- Los resultados casi perfectos son razonables **para este simulador**, pero no deben extrapolarse a datos bancarios reales: allí el patrón no es tan nítido y habría que monitorear la deriva y reentrenar.
- El modelo se sirve mediante una **API REST en Flask** con un dashboard de resultados, una explicación del proceso y un simulador de transacciones.

1. Comprensión del negocio

Una entidad que procesa pagos necesita decidir, al momento de recibir una operación, si la autoriza o la retiene para revisión. Los dos errores posibles tienen costos distintos:

- **Falso negativo** (fraude no detectado): pérdida directa del monto, más costos de reclamo y exposición reputacional. Es el error más caro.
- **Falso positivo** (operación legítima retenida): costo de revisión manual y fricción con el cliente. Es tolerable en baja proporción.

De esta asimetría se derivan tres decisiones técnicas que atraviesan el trabajo:

- La **accuracy no sirve** como métrica: con una tasa de fraude menor al 1 %, un modelo que nunca predice fraude tendría más de 99 % de accuracy.
- La métrica de comparación es **PR-AUC** (área bajo la curva precisión-recall), que se concentra en la clase minoritaria y no se infla con los verdaderos negativos como la ROC-AUC.
- El umbral de decisión se elige maximizando **F2**, que pondera el recall el doble que la precisión.
- El modelo sólo puede usar información **conocida antes de autorizar**: no se usan los saldos posteriores a la transacción.

2. Comprensión de los datos

El archivo contiene 1.362.621 filas y 11 columnas útiles: *step* (hora de simulación, 1 a 743), *type*, *amount*, identificadores de origen y destino, saldos antes y después para ambas cuentas, la etiqueta *isFraud* y la columna *isFlaggedFraud*, que la consigna indica no utilizar.
La auditoría estructural no encontró valores faltantes (0) ni filas duplicadas (0).

2.1 Tipo de transacción

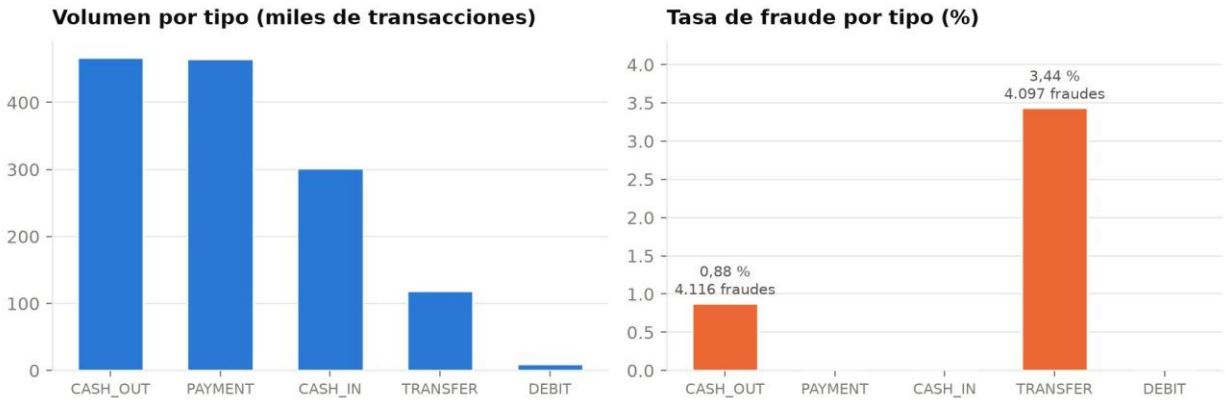


Figura 1. Volumen y tasa de fraude por tipo de transacción.

El fraude aparece **sólo en TRANSFER y CASH_OUT**. Aun así se conservan todos los tipos: eliminarlos cambiaría el universo de decisión y el modelo debe saber reconocer como legítimas las operaciones de otros tipos.

2.2 Monto

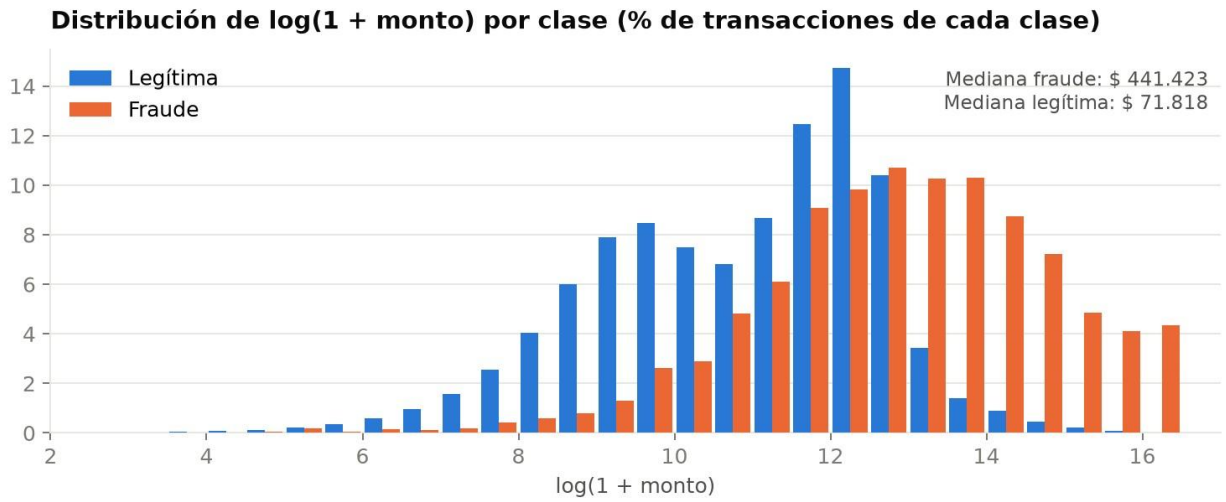


Figura 2. Distribución de log(1 + monto) por clase.

El monto es muy asimétrico. Los fraudes son sensiblemente mayores (mediana $ 441.423 contra $ 71.818 en legítimas; percentil 99 de $ 10.000.000, el tope del simulador). Los valores extremos **no se eliminan**, porque son precisamente señal. Se usa la transformación logarítmica para reducir la asimetría.

2.3 Hora del día

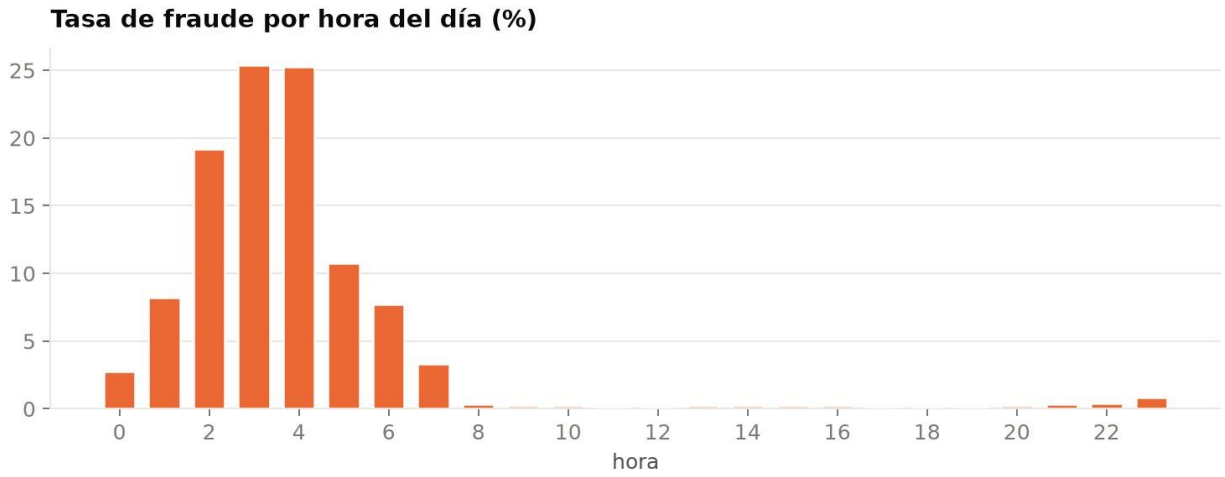


Figura 3. Tasa de fraude por hora (ventana con ambas clases).

El fraude se distribuye de forma pareja a lo largo del día, mientras que el tráfico legítimo cae de madrugada. Por eso la tasa por hora llega al 25,4 % a las 3 h. La hora del día es una variable útil y se calcula como *(step − 1) mod 24*.

2.4 Saldos y el patrón de vaciado

Los saldos en cero son frecuentes (32,1 % de los saldos de origen previos), por lo que no se tratan como faltantes. El hallazgo más relevante es que el **97,6 % de los fraudes extrae exactamente todo el saldo disponible** de la cuenta de origen (*amount = oldbalanceOrg*). De las 8.019 operaciones con ese comportamiento, el 99,99 % son fraude. Es coherente con el negocio (quien toma control de una cuenta intenta vaciarla), pero en datos reales sería mucho menos nítido: es una característica del simulador PaySim con que se generó el dataset.

2.5 Hallazgo de calidad: muestreo no uniforme

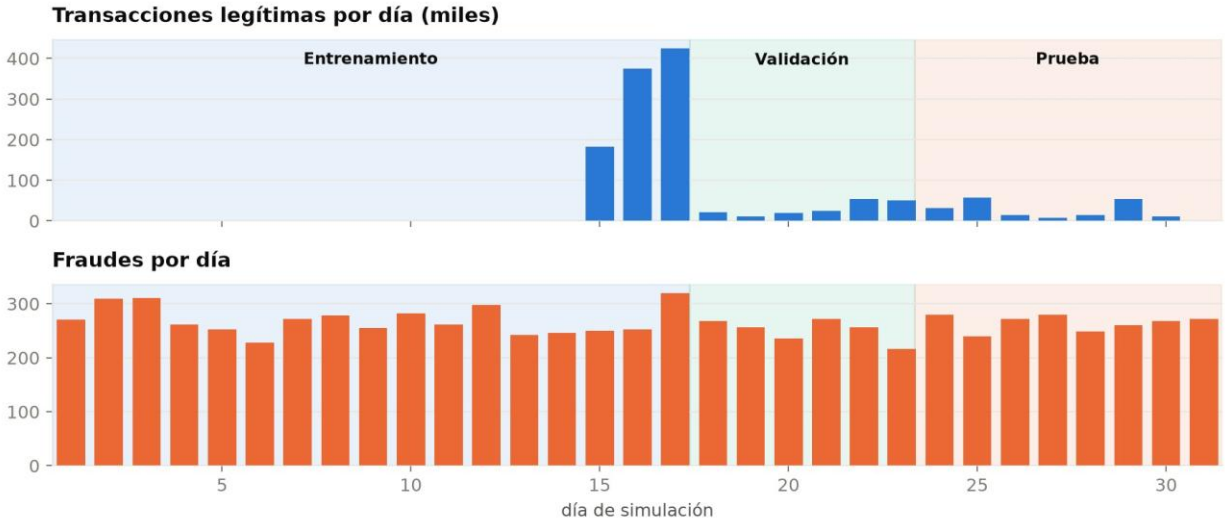


Figura 4. Legítimas y fraudes por día, con los períodos de entrenamiento, validación y prueba.

La columna de índice exportada (*Unnamed: 0*) revela que el archivo es el **tramo final** de una simulación más grande, al que se le agregaron **todos los fraudes** previos. Por eso las transacciones legítimas sólo existen entre los steps 353 y 718: los steps anteriores contienen 3.935 fraudes y ninguna legítima, y los posteriores, 296 fraudes. La tasa global de 0,60 % está inflada: dentro de la ventana con ambas clases es **0,29 %**.

> **Cómo se trató.** (1) No se usan *step* ni el día como variables, para que el modelo no aprenda “temprano = fraude”. (2) Se verificó que esos fraudes son representativos: vacían la cuenta en un 98,1 % de los casos (contra 97,4 % en la

ventana) y tienen la misma distribución horaria; por eso se conservan para entrenar. (3) La evaluación se informa también restringida a la ventana con ambas clases. (4) Un modelo entrenado sin esos fraudes logra PR-AUC 0,9992: el resultado no depende del artefacto.

2.6 Identificadores

nameOrig tiene prácticamente un valor distinto por fila, y *nameDest* más del 90 %. Codificarlos llevaría a memorizar cuentas en lugar de aprender comportamientos. Se descartan y sólo se conserva el prefijo del destino (C = cliente, M = comercio).

3. Preparación de los datos

Se reutilizó la ingeniería de variables de la etapa de preparación (notebook incluido en el repositorio). Se agregaron las dos variables de vaciado y se separaron dos conjuntos de variables. El escenario **A** (desplegado) usa sólo información previa a la autorización. El escenario **B** agrega saldos posteriores y errores contables y se evaluó sólo como control de fuga de información. Todas las transformaciones están en *src/features.py*, que comparten el entrenamiento y la aplicación web.

Variable	Definición	Justificación
type	Tipo de operación (One-Hot)	El fraude sólo ocurre en dos tipos
destPrefix	Prefijo del destinatario C/M (One-Hot)	Los comercios (M) nunca reciben fraude
amount, logAmount	Monto y log(1 + monto)	Los fraudes son de mayor monto; el log reduce la asimetría
hourOfDay, isNight	Hora del día y franja 0–5 h	La tasa de fraude es mucho mayor de madrugada
oldbalanceOrg, oldbalanceDest	Saldos previos	Conocidos antes de autorizar
origZeroBefore, destZeroBefore	Indicadores de saldo previo cero	Los ceros son frecuentes y tienen significado propio
amountToOrigBalance, amountToDestBalance	Monto / (saldo + 1)	Escala el monto respecto de la capacidad de la cuenta
amountMinusOrigBalance	Monto − saldo de origen	Distancia al vaciado total
drainsOrigin	1 si el monto = saldo de origen (> 0)	Patrón presente en el 97,6 % de los fraudes

Se excluyeron *isFlaggedFraud* (indicación de la consigna), *Unnamed: 0* (índice), los identificadores crudos, *step* y *day* (en una partición temporal el modelo vería días que nunca existieron en el entrenamiento, además del artefacto de muestreo) y los saldos posteriores. Las variables numéricas se escalan con **RobustScaler** (mediana y rango intercuartílico), menos sensible a las colas extremas de montos y saldos que la estandarización clásica. Las categóricas se codifican con **One-Hot Encoding**. El preprocesamiento vive dentro de un *Pipeline* de scikit-learn, que se ajusta sólo con los datos de entrenamiento.

4. Estrategia de validación

Se mantuvo la **partición temporal** definida en la etapa de preparación. Una partición aleatoria mezclaría pasado y futuro y daría una estimación optimista. En producción el modelo siempre se entrena con datos anteriores a los que va a evaluar.

Conjunto	Steps	Filas	Fraudes	Tasa	Uso
train	1–405	959.202	4.549	0,47 %	Ajuste de modelos

validation	406–548	201.372	1.496	0,74 %	Selección de modelo y umbral
test	549–743	202.047	2.168	1,07 %	Evaluación final (una vez)

El desbalance se trató con **ponderación de clases** (*class_weight*) en lugar de sobremuestreo (SMOTE) o submuestreo. La ponderación no fabrica ni descarta observaciones y no altera los conjuntos de validación y prueba. Además, con más de 4.500 fraudes en entrenamiento no hace falta generar ejemplos sintéticos. El conjunto de prueba se usó **una sola vez**, después de fijar el modelo y el umbral.

5. Modelado

Se compararon cuatro algoritmos de familias distintas, de menor a mayor complejidad, para poder justificar la elección por desempeño y no por costumbre. Todos se entrenaron con el mismo pipeline de preprocesamiento y el mismo conjunto de variables.

Algoritmo	Por qué se incluye	Configuración
Regla base	Referencia sin aprendizaje: si un modelo no la supera, no se justifica su complejidad	drainsOrigin = 1 → fraude
Regresión Logística	Lineal e interpretable; mide cuánto se logra con relaciones simples	class_weight balanced, C = 1
Árbol de Decisión	Reglas no lineales explicables al negocio	profundidad 8, hoja mínima 20, balanced
Random Forest	Bagging de árboles: reduce la varianza y es robusto a outliers	150 árboles, profundidad 14, balanced_subsample
Gradient Boosting (HistGB)	Boosting: corrige errores en forma secuencial; muy eficiente en tablas grandes	hasta 300 iteraciones, early stopping, balanced

Para cada modelo se eligió en validación el umbral que maximiza F2. Con ese umbral fijo se midió el desempeño en prueba. Además, cada algoritmo se reentrenó **sin las variables de vaciado** (ablación) para medir cuánto depende de ese único patrón: es la evidencia de robustez que se usó como criterio de desempate.

6. Evaluación y selección

6.1 Comparación de modelos

Modelo	PR-AUC valid.	PR-AUC prueba	Recall	Precisión	F2	FN	FP	PR-AUC sin vaciado
Regla base	0,9768	0,9699	96,96 %	100,00 %	0,9755	66	0	—
Regresión Logística	0,9983	0,9996	99,49 %	99,95 %	0,9958	11	1	0,6534
Árbol de Decisión	0,9999	0,9999	99,45 %	99,72 %	0,9950	12	6	0,5041
Random Forest	1,0000	1,0000	99,91 %	100,00 %	0,9993	2	0	0,9972
Gradient Boosting (HistGB)	**1,0000**	**1,0000**	**99,86 %**	**100,00 %**	**0,9989**	**3**	**0**	**0,9993**

FN = fraudes no detectados; FP = operaciones legítimas marcadas como fraude. Métricas de prueba con el umbral elegido en validación.

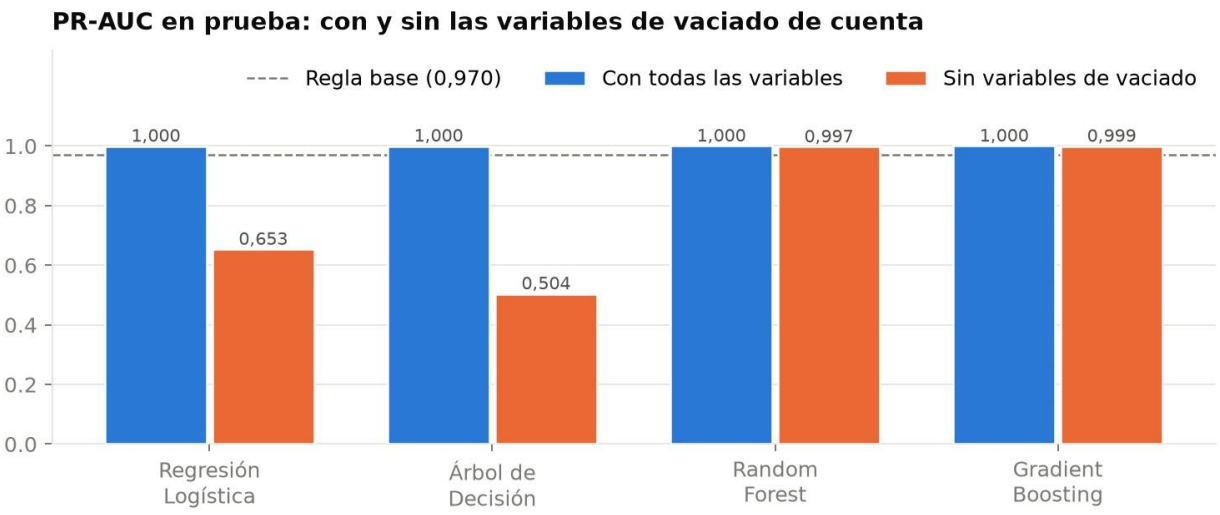


Figura 5. Robustez: PR-AUC en prueba con y sin las variables de vaciado.

Regla de selección. (1) Mayor PR-AUC en validación. (2) Ante empate, mayor PR-AUC en la ablación sin variables de vaciado (robustez). (3) Ante nuevo empate, el modelo más simple. Random Forest y Gradient Boosting (HistGB) empataron en el primer criterio. Gradient
Boosting (HistGB) se impuso en el segundo: sin las variables de vaciado mantiene PR-AUC
0,9994 en validación, frente a 0,9969 de Random Forest. En cambio, la Regresión Logística (0,651) y el Árbol (0,498) colapsan, porque dependen casi por completo de esa señal. Es decir, el boosting aprende el patrón de vaciado a partir del monto y el saldo aunque no se le dé explícitamente. Como ventaja operativa adicional, entrena en 2,5 s y su artefacto pesa 131 KB.

6.2 Umbral y matriz de confusión

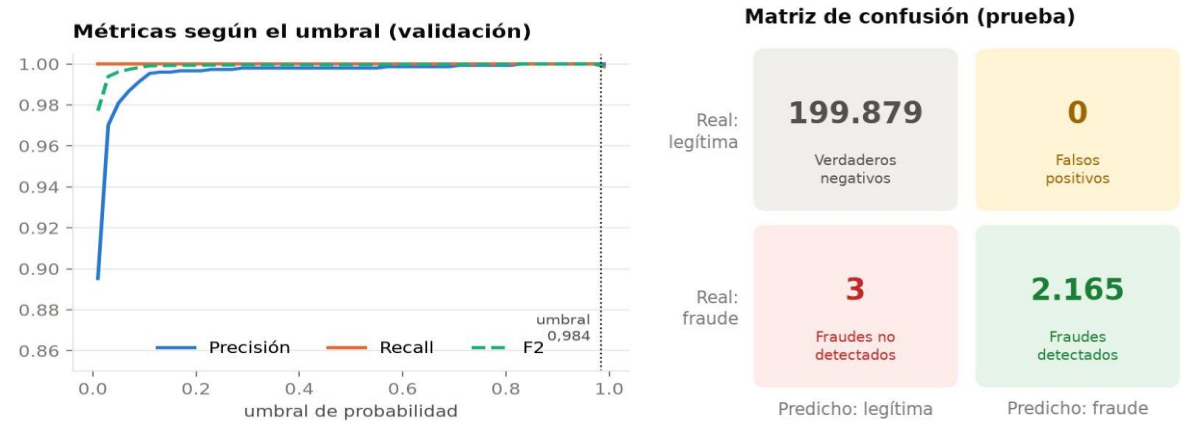


Figura 6. Métricas según el umbral (validación) y matriz de confusión del modelo elegido (prueba).

El umbral que maximiza F2 en validación es **0,984**. La meseta de la curva muestra que el resultado no es sensible al valor exacto. En prueba, el modelo no genera falsos positivos y deja pasar 3 fraudes. Ninguno de ellos vacía la cuenta de origen (mediana de $ 399.045): son los casos que escapan al patrón dominante, coherente con lo esperado.

6.3 Importancia de variables



Figura 7. Importancia por permutación: caída de PR-AUC al desordenar cada variable (escala logarítmica).

drainsOrigin domina con amplitud. Le siguen la relación monto/saldo del destino, el tipo de operación y los saldos de origen. La lectura es consistente con el análisis exploratorio, lo que da confianza en que el modelo aprende relaciones con sentido de negocio y no ruido.

6.4 Controles adicionales

Control	Resultado	Lectura
Prueba sólo en la ventana con ambas clases	1.870/1.872 detectados, 0 FP	El desempeño no se debe a los fraudes aislados del final
Entrenar sin el período sólo-fraude	PR-AUC 0,9992 (614 fraudes)	El artefacto de muestreo no es la fuente del resultado
Escenario B (con saldos posteriores)	PR-AUC 1,0000, 9 FN	No mejora al A: no hay fuga por información posterior
Regla base	Recall 97,0 %, 66 FN	El modelo recupera 63 fraudes adicionales

7. Razonabilidad de los resultados y limitaciones

Una PR-AUC de 1,000 debe despertar sospechas antes que celebración. Por eso se buscó activamente una explicación:

- **La causa es identificable y tiene sentido de negocio.** El fraude simulado vacía la cuenta de origen y casi ninguna operación legítima lo hace. Ese patrón solo explica el 97 % del resultado, como muestra la regla base.
- **No hay fuga de información.** El modelo desplegado no usa saldos posteriores, identificadores ni el step. El escenario con saldos posteriores no rinde más, y los controles sobre el artefacto de muestreo no cambian la conclusión.
- **La validación es temporal.** El modelo se evalúa sobre días futuros que nunca vio.

Limitaciones:

- Los datos son **sintéticos**. En un entorno bancario real los defraudadores adaptan su comportamiento, y no cabe esperar métricas de este nivel. El valor del trabajo está en la metodología, que es trasladable.
- El archivo es una muestra no uniforme, por lo que la prevalencia y la deriva temporal observadas están parcialmente distorsionadas.
- No se dispone del costo real de cada error. Si el negocio lo cuantifica, el umbral debería elegirse minimizando el costo esperado en lugar de maximizar F2.
- Sin historial por cliente no se pueden construir variables de comportamiento (velocidad de operaciones, montos habituales), que en la práctica son las más valiosas.

8. Aplicación web

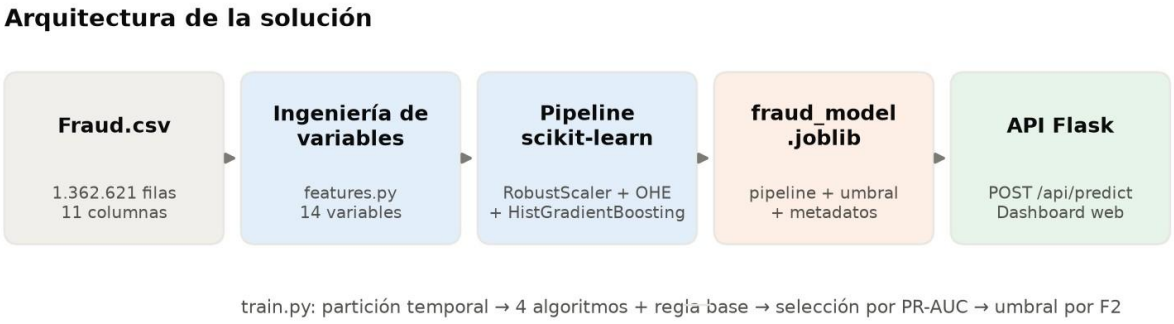


Figura 8. Arquitectura de la solución.

El pipeline completo (preprocesamiento, modelo y umbral) se serializa con *joblib* y lo carga una aplicación **Flask**, publicada en https://caece-aprendizaje-artificial-ezequiel.onrender.com (Render, servidor gunicorn). Ofrece:

- **Resultados**: indicadores clave, comparación de modelos, robustez, matriz de confusión, umbral e importancia de variables.
- **Proceso**: recorrido CRISP-DM con los gráficos exploratorios interactivos y las decisiones tomadas.
- **Simulador**: formulario que consulta la API con una transacción (o con ejemplos reales del período de prueba) y muestra la probabilidad, la decisión sugerida y las señales que la contextualizan.
- **API REST**: *POST /api/predict*, *GET /api/metrics* y *GET /api/health*, con validación de entradas.

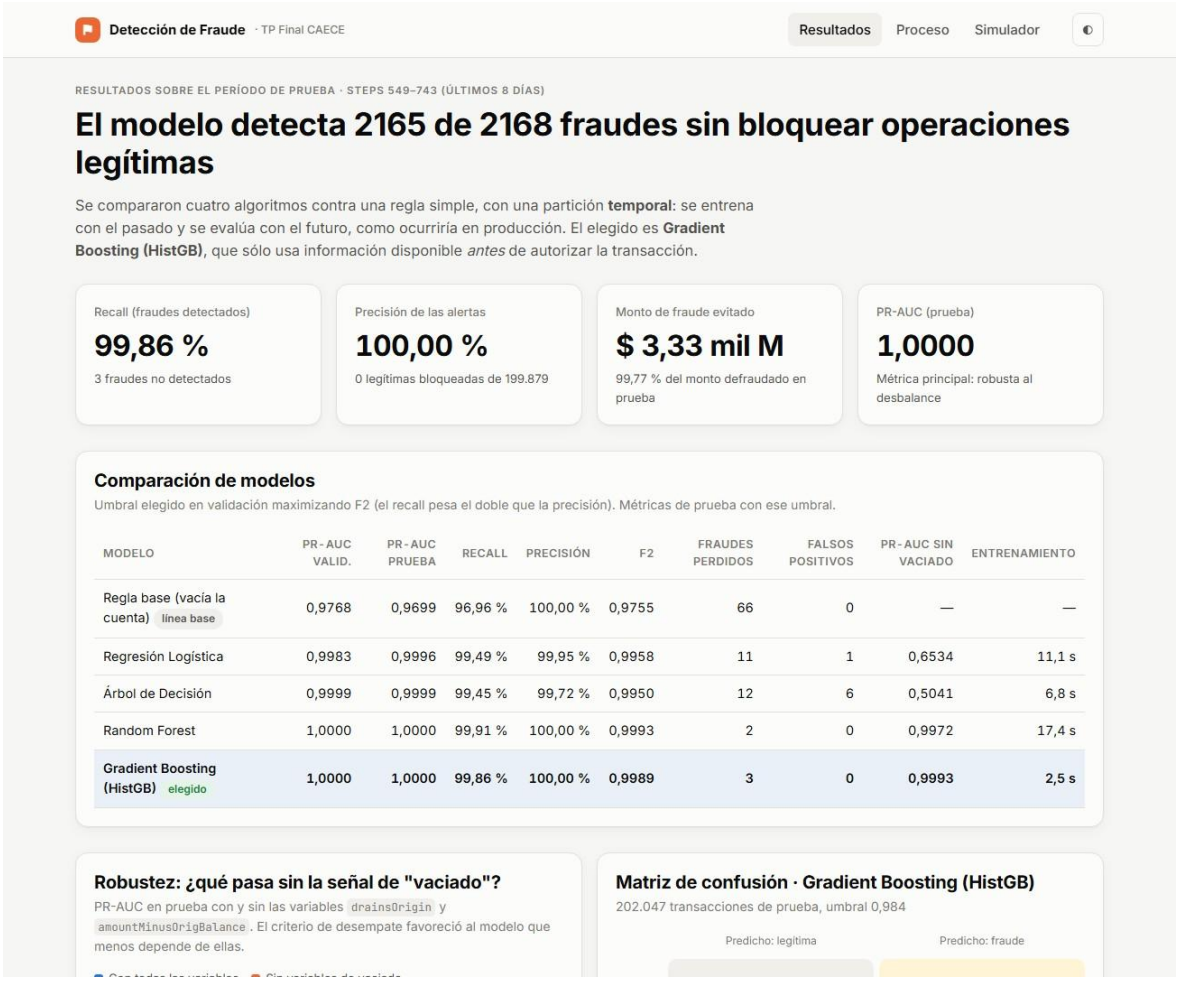


Figura 9. Página de resultados.

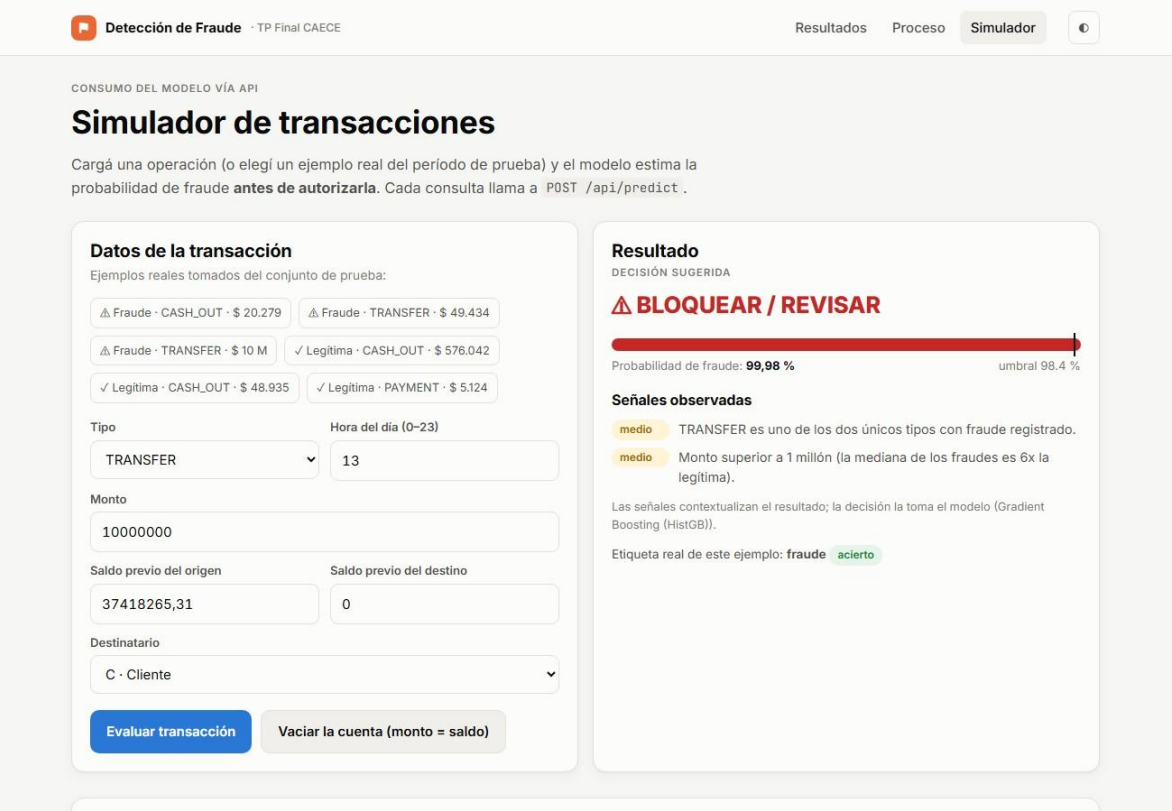


Figura 10. Simulador: un fraude real de $ 10 M que no vacía la cuenta y el modelo detecta igual.

Ejemplo de consumo de la API:

```
curl -X POST http://localhost:5000/api/predict -H "Content-Type: application/json" \   -d
'{"type":"TRANSFER","amount":181000,"oldbalanceOrg":181000,"oldbalanceDest":0,"hourOfDay":3
,"destPrefix":"C"}'

{"probabilidad": 0.9999, "umbral": 0.984, "es_fraude": true, "decision": "BLOQUEAR / REVISAR",
"senales": [...]}
```

9. Conclusiones

- Se construyó un modelo de **Gradient Boosting (HistGB)** que, con información disponible antes de autorizar, detecta el 99,86 % de los fraudes del período de prueba sin bloquear operaciones legítimas.
- La elección del algoritmo se basó en tres criterios explícitos: desempeño (PR-AUC), robustez (ablación) y simplicidad. El umbral se fijó por F2, acorde al mayor costo del fraude no detectado.
- El análisis crítico permitió detectar dos características del dataset que condicionan la lectura de los resultados: el patrón de vaciado de cuenta y el muestreo no uniforme. Ambas se cuantificaron y se controlaron.
- La solución es reproducible de punta a punta (*src/train.py*) y está disponible como aplicación web y API.

Trabajo futuro

- Definir con el negocio el costo de cada tipo de error y optimizar el umbral por costo esperado.
- Incorporar variables de comportamiento por cuenta (frecuencia, montos habituales, antigüedad) con datos reales.
- Monitorear la deriva (tasa de alertas, distribución de scores) y reentrenar periódicamente.
- Agregar explicaciones por predicción (por ejemplo, valores SHAP) para los analistas que revisan las alertas.

Anexo A. Reproducibilidad

Código fuente: https://github.com/ezediazf/tpfinal. El dataset no se incluye en el repositorio por su tamaño (120 MB); se descarga desde el enlace de la consigna y se guarda como *data/Fraud.csv*.

```
pip install -r requirements.txt
python src/train.py # entrena, compara y guarda models/ python app/app.py
# http://127.0.0.1:5000 python docs/build_report.py # regenera este
informe python docs/build_slides.py # regenera la presentación
```

Ruta	Contenido
notebooks/	Etapa de preparación de datos (auditoría, EDA, variables, partición)
src/features.py	Ingeniería de variables compartida por entrenamiento y app
src/train.py	Partición temporal, entrenamiento, comparación, selección, métricas
models/	fraud_model.joblib (pipeline + umbral) y metrics.json (resultados)
app/	Aplicación Flask: API, dashboard, proceso y simulador
docs/	Informe PDF, presentación HTML, figuras y scripts que los generan

Semilla aleatoria: 42. Entorno: Python, scikit-learn, pandas, Flask. Métricas generadas el 2026-10-19 08:11.